Agentic SDLC Documentation & Design Platform

Project concept and requirements specification | Version 1.0 | 7 November 2026

Status: proposed project. This document describes intended capabilities, not an implemented or validated system.

1. Problem Statement

Software teams receive assignments, client briefs and requirements documents in inconsistent formats. Engineers must interpret the business problem, identify missing information, model entities and workflows, define interfaces, design tests and prepare multiple engineering artifacts. Producing these first drafts manually is time-consuming, repetitive and vulnerable to inconsistencies between documents. Smaller teams and time-constrained engagements may have limited capacity for thorough documentation.

The proposed solution uses a coordinated set of AI agents to analyze an uploaded assignment and generate a traceable, reviewable package of software development lifecycle (SDLC) artifacts. It aims to reduce drafting effort and accelerate early design and test planning without replacing engineering judgment, client approval or verification.

2. Vision and Objectives

- Accept a software assignment from different clients or industries, in PDF, DOCX or plain text.
- Extract stated requirements, constraints, actors, business rules and acceptance criteria, with references to the source.
- Flag ambiguity, missing information and proposed assumptions separately from confirmed requirements.
- Generate a coherent set of editable, downloadable engineering documents and diagrams.
- Enable human review, corrections, regeneration and approval before export.
- Measure time saved, artifact completeness and reviewer corrections rather than claiming unverified productivity gains.

3. Users and Main Scenario

Primary users: software engineers, technical leads, solution architects, QA/test engineers, business analysts and project teams. The client or product owner is the authority for business-policy decisions.

Example: An engineer uploads a mortgage pricing exception assignment. The platform extracts request, review, revision and history requirements; identifies uncertain policies; proposes domain entities, APIs, diagrams and test scenarios; and produces an exportable package. The engineer reviews all assumptions and edits or approves the results.

4. Functional Requirements

- FR-01 Upload and parse PDF, DOCX and TXT assignments; reject unreadable or unsupported files with clear feedback.

- FR-02 Identify actors, use cases, functional/nonfunctional requirements, constraints and acceptance criteria.
- FR-03 Maintain a requirements register with unique IDs, source references, status and open questions.
- FR-04 Generate proposed domain entities, relationships, data dictionary and ER diagram.
- FR-05 Generate architecture overview, component responsibilities, design decisions and alternatives/trade-offs.
- FR-06 Generate workflow descriptions and sequence diagrams for key use cases.
- FR-07 Propose API endpoints, request/response examples, validation and error scenarios where an API is relevant.
- FR-08 Generate test strategy, test protocol, test cases/test-suite specification and requirements-to-tests traceability matrix.
- FR-09 Generate deployment/local execution guidance, security considerations and production-readiness checklist as proposals.
- FR-10 Show provenance, assumptions, uncertainty and consistency warnings; never silently treat inferred rules as confirmed.
- FR-11 Allow a reviewer to edit/approve sections and regenerate affected artifacts.
- FR-12 Export individual DOCX/PDF/Markdown artifacts and a ZIP package with a manifest.

5. Proposed Output Package

Category	Generated artifacts (proposed)
Requirements	Problem statement, scope, assumptions, use cases, functional/nonfunctional requirements, acceptance criteria
Architecture & design	Architecture document, component diagram, design decisions, domain model, ER diagram, data dictionary
Behavior & interfaces	Sequence diagrams, workflow/state diagrams, API specification and sample contracts
Quality assurance	Test strategy, test protocol, test-case suite specification, edge cases, traceability matrix
Delivery & operations	Local execution plan, deployment proposal, security/risk considerations, production-readiness checklist
Governance	Open-questions register, assumptions log, source-to-artifact traceability, AI-generation/review record

The platform should generate only artifacts relevant to the uploaded assignment. For example, it should not invent a microservices design or deployment platform when the brief does not justify one. Generated test documents are test plans/specifications, not proof that executable tests have passed.

6. Proposed Agentic Architecture (CrewAI)

A manager/orchestrator coordinates specialized agents. Each agent produces structured outputs validated against a schema; shared requirement IDs and an artifact manifest support consistency.

- Intake & Requirements Agent: parses input, extracts source-grounded requirements and flags ambiguity.
- Architecture & Domain Agent: proposes solution boundaries, entities, relationships and trade-offs.
- Workflow & API Agent: defines interactions, sequence flows and interface contracts.

- QA Agent: derives test scenarios, protocol, negative cases and traceability from approved requirements.
- Consistency/Review Agent: checks contradictions, missing links, unsupported assumptions and incomplete artifacts.
- Document Publisher (deterministic service, not necessarily an agent): renders DOCX and diagrams, packages files for download.

High-level processing flow

Upload → Parse & extract → Requirements register → Human clarification/approval → Design & interface generation → QA generation → Cross-document validation → Human review → Downloadable package

Suggested stack: Python, CrewAI, Pydantic structured outputs, Gradio for the demo UI, python-docx for editable Word documents, Mermaid or PlantUML for diagram source and rendering, and a local file store for the prototype. A database and job queue can be added only when persistence or concurrent usage requires them.

7. Human Oversight and Safety

- Label every statement as source requirement, derived design proposal or unresolved assumption.
- Require human approval for business rules, architecture decisions and any security/compliance claims.
- Preserve source references and document version; make generated diagrams editable when practical.
- Treat uploaded documents as untrusted data; do not obey embedded instructions that attempt to control the system.
- Avoid sending confidential client material to unapproved external models; define retention and deletion behavior.
- Prevent fabricated test results, approvals, certifications or claims of production readiness.

8. Nonfunctional Requirements and Success Criteria

- Usability: upload, review and download through a simple browser UI.
- Reliability: failed agent steps produce actionable errors and allow safe reruns.
- Consistency: requirement IDs link designs and test cases; validation reports gaps.
- Portability: generated DOCX files and diagrams remain accessible without the platform.
- Observability: record job status, step timings, errors, model configuration and generation version without leaking confidential content.
- Evaluation: compare human drafting time, reviewer correction count, requirement coverage and artifact consistency against a manual baseline.

9. Scope and Delivery Plan

MVP — first demonstration

- Upload one PDF/DOCX assignment and extract a requirements register plus assumptions/open questions.
- Use 2–3 agents: Requirements, Design, and QA/Validation.
- Generate three downloadable Word documents: Requirements & Assumptions; Architecture & Domain Design; Test Plan & Cases.
- Produce one ER diagram and one sequence diagram from the extracted use cases.
- Provide a review screen and a ZIP export; demo using a nonconfidential assignment.

Later iterations

- Additional diagrams, API/OpenAPI generation, richer traceability and document templates.
- Reviewer comments, version history, incremental regeneration and multi-project workspace.
- Optional generation of starter code and executable test scaffolds, explicitly separated from verified results.
- Integration with GitHub, Jira or CI systems only after core artifact quality is demonstrated.

10. Acceptance Criteria for the MVP

- A user can upload a supported assignment and see extracted requirements with source references.
- The system visibly separates explicit requirements, assumptions and open questions.
- It produces valid downloadable DOCX files, an ER diagram and a sequence diagram.
- At least one test case is traceable to each selected core functional requirement.
- The consistency checker flags an intentionally introduced mismatch or unsupported assumption.
- A reviewer can inspect and approve the package; no generated output is represented as independently verified.
- The demo can be rerun with a second, different assignment to show domain independence.

11. Demonstration and Business Value

Demonstrate: upload an assignment → inspect extracted requirements and questions → show the agent workflow → review architecture and diagrams → inspect traceable test cases → download the documents. Compare the first drafts with a human-reviewed reference and show what still needs correction.

Value proposition: faster first-draft production, more consistent handoffs, earlier discovery of ambiguities and better requirements-to-test traceability. Actual time savings and quality improvements must be measured in a pilot; no percentage reduction is assumed.

12. Key Risks and Mitigations

- Hallucinated rules or invented interfaces → source references, explicit assumption labels and approval gates.
- Inconsistent artifacts → shared requirement IDs, structured schemas and cross-artifact validation.
- Poor diagram quality → constrained templates and rendering checks.
- Sensitive client data exposure → approved model endpoints, access controls and retention policies.
- Excessive scope → deliver the MVP before adding more agents or code generation.

13. Next-Day Implementation Checklist

- Create a Python repository with separate app, agents, schemas, exporters and tests folders.
- Build a file upload/parser and define structured requirements/assumptions models.
- Implement the Requirements Agent and validate output on one sample assignment.
- Implement Design and QA agents with requirement IDs carried through.
- Export Word files and render diagrams; add download buttons and a ZIP package.
- Run two sample assignments, inspect outputs manually and record limitations.